

# DOCUMENT TECHNIQUE

Projet 68 — Plateforme de programme de fidélisation

*Fidélité ESP*

Master CCA — École Supérieure Polytechnique de Dakar  
Année universitaire 2025–2026

Auteur : **Bineta DIAGNE**

## 1. Introduction

---

Ce document présente l'architecture technique, le modèle de données et les conventions de code retenus pour la réalisation du Projet 68 : une plateforme web de programme de fidélisation permettant la gestion des adhérents, l'attribution automatisée de points, l'évolution entre paliers de statut, et l'échange de récompenses.

La plateforme a été développée avec la stack imposée par le cahier des charges : Apache, MySQL, PHP 8 et PhpMyAdmin (environnement XAMPP), avec un front-end en HTML5, CSS3 et JavaScript natif.

## 2. Modèle de données

---

### 2.1 Modèle Conceptuel de Données (MCD) — vue synthétique

La base de données fidelite\_esp comprend 7 entités principales, reliées par des clés étrangères :

| Table               | Rôle                                                                                   |
|---------------------|----------------------------------------------------------------------------------------|
| utilisateurs        | Comptes de connexion : email, mot de passe haché, rôle (admin/gestionnaire/client)     |
| paliers             | Les 4 statuts du programme (Bronze/Argent/Or/Platine), seuil de points, multiplicateur |
| adherents           | Profil fidélité d'un client : points, palier, informations personnelles                |
| recompenses         | Catalogue des récompenses échangeables, avec coût en points et stock                   |
| transactions_points | Historique de tous les mouvements de points (achats, bonus, ajustements)               |
| echanges            | Demandes d'échange de points contre une récompense, avec workflow de validation        |
| audit_log           | Journal horodaté de toutes les actions sensibles                                       |

### 2.2 Relations entre les entités

- adherents.palier\_id → paliers.id (chaque adhérent appartient à un palier)
- adherents.utilisateur\_id → utilisateurs.id (lien optionnel vers un compte de connexion)
- adherents.parraine\_par → adherents.id (auto-référence pour le parrainage)
- transactions\_points.adherent\_id → adherents.id
- transactions\_points.cree\_par → utilisateurs.id (traçabilité de l'auteur)
- echanges.adherent\_id → adherents.id
- echanges.recompense\_id → recompenses.id
- echanges.traite\_par → utilisateurs.id
- audit\_log.utilisateur\_id → utilisateurs.id

### 2.3 Règles de gestion associées au modèle

- Le palier d'un adhérent est recalculé automatiquement à partir de la somme de ses points positifs sur les 12 derniers mois glissants (fonction recalculer\_palier()).
- Le solde de points disponibles est recalculé à partir de l'historique complet des transactions, moins les points réservés par des échanges en attente, validés ou livrés (fonction recalculer\_soldes()).
- Un échange refusé restitue automatiquement les points au solde disponible de l'adhérent.
- Toute action sensible (création, modification, suppression, connexion, validation d'échange) est journalisée dans audit\_log.

## 3. Architecture applicative

---

### 3.1 Organisation des dossiers

|                          |                                                           |
|--------------------------|-----------------------------------------------------------|
| fidelite-app/            |                                                           |
| — config/                | → connexion PDO, configuration générale, sessions         |
| — includes/              | → authentification, fonctions communes, layout, SimplePDF |
| — adherents/             | → CRUD adhérents                                          |
| — transactions/          | → attribution et historique des points                    |
| — recompenses/           | → catalogue de récompenses (CRUD)                         |
| — echanges/              | → demandes d'échange et workflow de validation            |
| — paliers/               | → gestion des paliers de statut (CRUD)                    |
| — export/                | → exports PDF et CSV                                      |
| — assets/                | → CSS, JavaScript                                         |
| — sql/                   | → script de création de la base + seed des comptes        |
| — dashboard.php          | → tableau de bord (staff)                                 |
| — mon-compte.php         | → espace personnel (client)                               |
| — login.php / logout.php |                                                           |
| — audit.php              | → journal d'audit (admin)                                 |

### 3.2 Modèle applicatif en couches

- Couche présentation : fichiers .php mêlant HTML et PHP, avec includes/header.php, sidebar.php, footer.php pour la mise en page commune
- Couche accès aux données : PDO avec requêtes préparées, centralisé dans config/db.php
- Couche logique métier : fonctions réutilisables dans includes/functions.php (calcul des paliers, soldes, pagination, audit)
- Couche sécurité transverse : includes/auth.php, appelé en tête de chaque page protégée

### 3.3 Rôles et contrôle d'accès

| Rôle         | Périmètre d'accès                                                                |
|--------------|----------------------------------------------------------------------------------|
| admin        | Accès complet, y compris suppression et journal d'audit                          |
| gestionnaire | Gestion quotidienne (adhérents, points, récompenses, échanges), sans suppression |
| client       | Consultation de son profil, du catalogue, demande d'échanges                     |

## 4. Sécurité

- Mots de passe hachés avec password\_hash() (algorithme bcrypt) et vérifiés avec password\_verify() — jamais de mot de passe en clair stocké ou manipulé
- Requêtes préparées PDO systématiques (PDO::ATTR\_EMULATE\_PREPARES désactivé) contre les injections SQL
- Échappement systématique à l'affichage via la fonction h() (htmlspecialchars) contre les attaques XSS
- Jeton CSRF unique par session, vérifié sur chaque formulaire de modification (hash\_equals())
- Sessions avec expiration automatique après 30 minutes d'inactivité et régénération de l'identifiant de session à la connexion (protection contre la fixation de session)
- Contrôle d'accès par rôle systématique en tête de chaque script (exiger\_role())
- Journal d'audit horodaté de toutes les actions sensibles avec adresse IP

## 5. Conventions de code

- Noms de fichiers, fonctions et variables en français, en minuscules avec underscore (snake\_case), pour une cohérence avec le vocabulaire métier du cahier des charges
- Chaque page protégée commence systématiquement par : require config/app.php, config/db.php, includes/auth.php, includes/functions.php, puis `exiger_role([...])`
- Toute sortie de donnée dynamique dans le HTML passe par la fonction `h()`
- Toute requête SQL utilise `PDO::prepare()` + `execute()`, jamais de concaténation directe de variable
- La logique de recalcul des soldes et des paliers est centralisée dans `includes/functions.php` pour éviter toute duplication et incohérence
- Les fichiers de configuration sensibles (`config/db.php`) sont isolés du reste du code pour faciliter le changement d'environnement

## 6. Choix techniques particuliers

---

### 6.1 Génération des mots de passe de démonstration

Les mots de passe ne pouvant pas être hachés de façon fiable directement dans un script SQL, un script PHP séparé (`sql/seed_users.php`) crée les comptes de démonstration après l'import de la base, en utilisant la véritable fonction `password_hash()` de PHP.

### 6.2 Génération des exports PDF

Le cahier des charges recommande l'usage de TCPDF, FPDF ou DOMPDF. Ces bibliothèques nécessitant une installation via Composer, une petite classe PHP native (`includes/SimplePDF.php`) a été développée pour générer des documents PDF valides sans dépendance externe, afin de garantir le fonctionnement immédiat du projet sur tout poste XAMPP, y compris sans accès Internet. Sa structure interne suit les spécifications du format PDF (objets, xref, flux de contenu) et a été validée avec l'outil `qpdf --check`.